

Relazione-6a

mpirun :

First I entered the container and look if the compiler of mpi exist with the command
mpirun --version

```
root@wn1:~/6a-MPI/mpi/base# mpirun --version
mpirun (Open MPI) 4.1.6

Report bugs to http://www.open-mpi.org/community/help/
root@wn1:~/6a-MPI/mpi/base# |
```

We can see that there exist the version 4.1.6 that is a good start.

Now lets work on the exercise given on the homework:

→ mpirun -n 4 date # crea 4 task sull'host locale in cui viene eseguito date.

```
root@wn1:~/6a-MPI/mpi/base# mpirun -n 4 date
Fri Apr 24 09:25:51 UTC 2026
Fri Apr 24 09:25:51 UTC 2026
Fri Apr 24 09:25:51 UTC 2026
Fri Apr 24 09:25:51 UTC 2026
root@wn1:~/6a-MPI/mpi/base#
```

(That means we want 4 task to run the date of today)

→ mpirun -host wn1:4,wn2:2 -n 5 hostname

```
root@wn1:~/6a-MPI/mpi/base# mpirun -host wn1:4,wn2:2 -n 5 hostname
wn1
wn1
wn1
wn2
wn1
root@wn1:~/6a-MPI/mpi/base# |
```

(This means use wn1 with 4 slots and wn2 with 2 slots and we have 5 tasks disponibile)

→ `mpirun -hostfile hostfile.txt -n 2 hostname`

```
root@wn1:~/6a-MPI/mpi/base# mpirun -hostfile hostfile.txt -n 5 hostname
wn1
wn1
wn1
wn2
wn1
root@wn1:~/6a-MPI/mpi/base# cat hostfile.txt
#wn21
#wn22
wn1 slots=4
wn2 slots=4
root@wn1:~/6a-MPI/mpi/base#
```

Here we are making use of a file called `hostfile.txt` where we have the number of slot to use. Using the command `cat hostfile.txt` you can see it.

→ `mpirun -n 6 -hostfile hostfile.txt hostname`

```
root@wn1:~/6a-MPI/mpi/base# mpirun -n 6 -hostfile hostfile.txt hostname
wn1
wn1
wn1
wn2
wn2
wn1
root@wn1:~/6a-MPI/mpi/base#
```

`mpicc:`

This compiler is the same like `gcc` put with openMPI directives in plus

Now lets work with `mpicc` on the CLUSTER with SLURM

This is the output of the `mpi1_helloworld.slurm` :

```
module
#SLURM_JOB_NODELIST      : wn[21-22]
#SLURM_NNODES            : 2
#SLURM_NTASKS            : 4
#SLURM_JOB_CPUS_PER_NODE : 2(x2)
Hello from task 0 on wn21!
Hello from task 1 on wn21!
MASTER: Number of MPI tasks is: 4
Hello from task 2 on wn22!
Hello from task 3 on wn22!
```

We can see that the program used 2 working nodes and 4 tasks each node took 2 tasks.

Using sendrecv this is the output (That is sendrecv in a ring of task that is task 0 send something to task 1, task 1 send to 2 and it goes on like a ring).

```
→ base git:(main) x mpicc mpi4_sendrecv.c -o mpi4_sendrecv
→ base git:(main) x mpirun -np 3 mpi4_sendrecv
Process (2 % 3 = 2) receives 1 from process 1
Process (0 % 3 = 0) receives 2 from process 2
Process (1 % 3 = 1) receives 0 from process 0
```

So the basics are well understood.